

Accelerating LLM Inference: KV Cache Mechanisms and VRAM Compression Strategies

學號：B812111001 姓名：張博翔 日期：2026 / 06 / 18

【Abstract】 LLM 進行長對話時，生成新字皆須重新計算先前對話的 Key-Value，導致延遲隨對話長度呈二次方成長而面臨運算瓶頸。KV Cache 技術透過將已計算的 Key-Value 矩陣暫存於記憶體中，使模型無須重複運算以降低推論延遲。然而在長文本或多用戶並行的場景下，其佔用的 VRAM 可能高達數十 GB，衍生出 Quantization、Paged Attention、Sparse Attention 等壓縮技術。本報告探討 KV Cache 的運作機制及其與動態規劃「最優子結構」，並分析各壓縮策略的 Trade-off。

1. Review : KV Cache

1-1 AI 如何讀懂一段文字

現代大型語言模型（LLM）的核心是 Transformer 架構，Transformer 是一種專門處理語言的 Neural Network 設計，其中關鍵運算叫做 Self-Attention，能讓 AI 在讀每個字的時候自動參考前文、判斷哪些字和當前最相關。

每一個字（Token）在計算注意力時，會產生三個向量：

- Query (Q)：這個字想問什麼？(主動向其他字發問)
- Key (K)：這個字代表什麼？(被動等待其他字來比對)
- Value (V)：如果被選中，這個字代表資訊？(實際貢獻的語意資訊)

模型透過計算 Q 與所有 K 的相似度決定要注意哪些字，再把對應的 V 加權總合，就得到最終的輸出。

目前問題在於每次新增一個 Token 就必須把它與前面所有 Token 的 K、V 全部重新計算一遍，時間複雜度就會成長到 $O(n^2)$ 。

1-2 KV Cache

KV Cache 的解法非常直接，算過的 K 和 V 就暫存起來，下次直接用。

Cache 即為暫存技術，把耗時計算的中間結果先存在高速記憶體裡，需要時直接讀取，避免重複運算。

當模型生成第 n 個 Token 時，第 1 到 n-1 個 Token 的 K、V 矩陣早已計算完畢。KV Cache 將這些矩陣存放在 GPU 的 VRAM 中，下次生成第 n+1 個 Token 時，只需要計算新 Token 的 K、V，然後直接從快取讀取舊的 K、V，不必重算。這讓推論的時間複雜度從 $O(n^2)$ 降為 $O(n)$ ，大幅加快回應速度。

1-3 Flow Chart

下圖展示 KV Cache 在 LLM 推論時的完整運作流程：

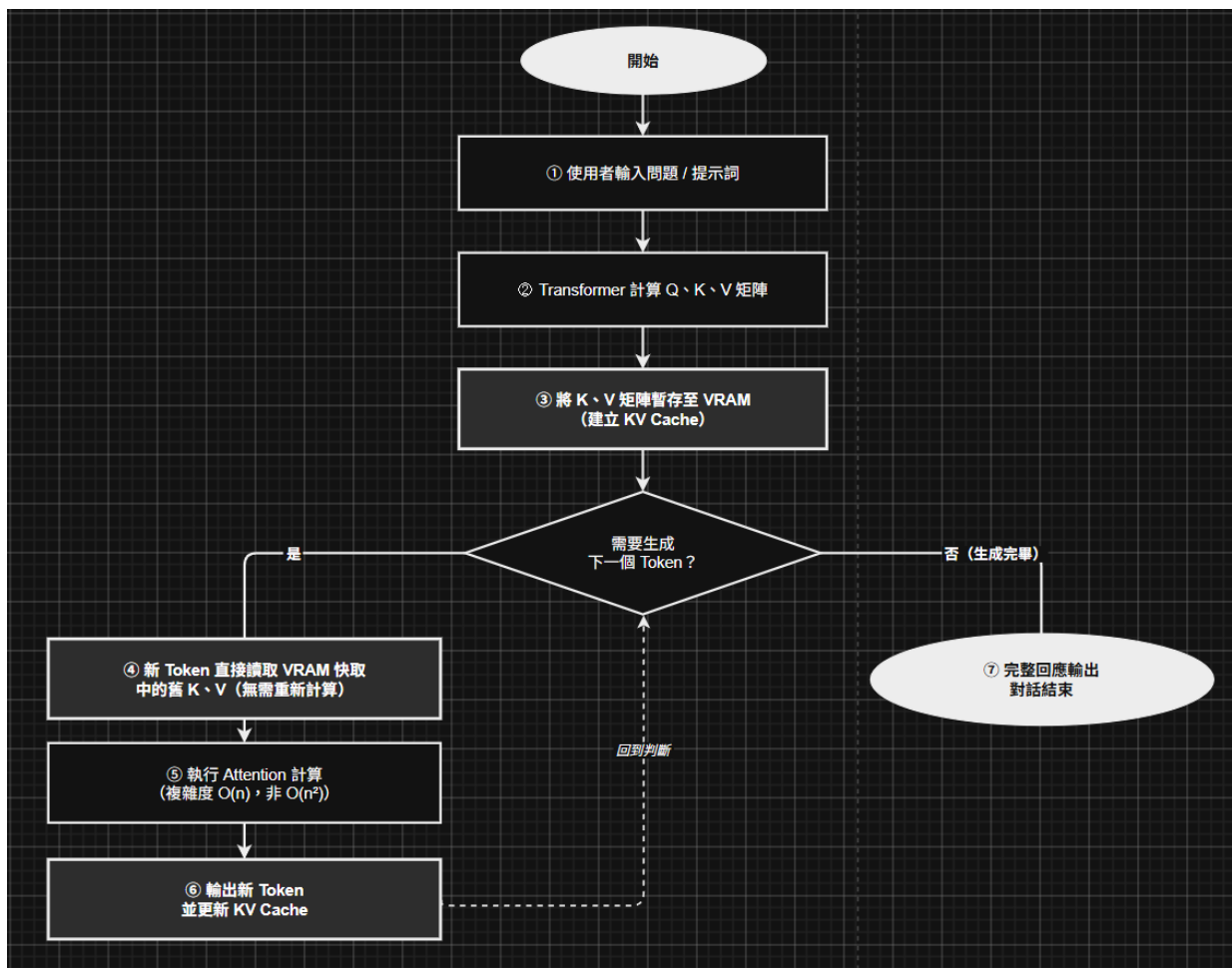


圖1：KV Cache 推論流程示意圖

1-4 與 Dynamic Programming 的關聯

課程連結

KV Cache 的設計思維與演算法中 Dynamic Programming 高度吻合。動態規劃的核心想法是把大問題拆成小問題，算過的小問題結果存起來，遇到一樣的問題就直接查，不需重複算。這個策略成立的前提有兩個條件：

- **最優子結構**：大問題的最佳解，可以由小問題的最佳解組合而成
- **重疊子問題**：同樣的小問題會被反覆遇到，值得存起來重用

KV Cache 正好符合這兩個條件，每個 Token 的 K、V 矩陣一旦算出，在後續生成每一個新字時都會被反覆用到，因此模型將這些結果暫存之後直接查表讀取，而非重新求解。

「算一次、存起來、重複利用」的手法又可稱為 Memoization。KV Cache 本質上就是 Memoization 在深度學習推論階段的一個典型實踐。

2. Application

2-1 應用領域

以日常使用的對話型 AI 為例，當你和 AI 進行多輪對話時，表面上看起來 AI 好像一直「記得」

你說過的話，但背後其實是 KV Cache 在支撐這件事。每一輪對話新增的內容，AI 只需要計算那一輪的 K、V 並追加進快取，前面所有輪次的結果直接讀取即可。

A. 多輪對話：

一般人線上聊天時不需要把整個聊天記錄重新看一遍才能回覆，只要看最新那則，其他的印象還留在腦子裡。KV Cache 讓 AI 也能做到類似的事：

- 你傳第 1 則訊息 → AI 計算這則訊息所有字的 K、V，存入快取
- 你傳第 2 則訊息 → AI 只計算第 2 則的 K、V，第 1 則直接從快取讀取
- 以此類推，每次只處理新增內容，整體運算量隨對話輪數線性成長，而非平方成長

例如文件摘要與分析：

當你有一份長篇文件，例如合約、論文、會議記錄等丟給 AI 請它分析時，AI 需要先讀完整份文件。有了 KV Cache，這份文件的 K、V 只需要計算一次並存入快取，之後你不管問幾個問題如「幫我抓重點」、「第三段在說什麼」、「有沒有法律風險」，AI 都不需要重新閱讀文件，直接從快取提取語境來回答。

B. 程式碼開發：

如 GitHub Copilot 這類工具，AI 需要持續參考整份程式碼的上下文才能給出符合邏輯的建議，KV Cache 讓 AI 能夠「記住」已經讀過的程式碼結構，每次你新增或修改幾行，AI 只需要處理那幾行的變動，不需要重新掃描整個專案。

C. 多模態輸入（圖片 + 文字）：

許多 AI 支援同時輸入圖片和文字，例如上傳一張電路圖然後問問題，圖片在 AI 眼中會被拆解成大量的 Token（AI 處理資訊的最小單位），計算量遠比純文字龐大。KV Cache 讓圖片的 K、V 只需計算一次，後續追問「這個元件是什麼」、「哪裡可能有問題」時，不需要每次重新解析圖片，大幅降低回應延遲。

D. System Prompt Cache：

企業在部署 AI 客服或專業助理時通常會在對話開始前預先輸入一大段「系統提示詞」，例如公司規範、產品知識庫、回答風格指引等。這段內容對每一位使用者都是一樣的，因此可以預先計算好 K、V 並永久快取，所有使用者共用同一份快取結果，省去重複計算的成本，也讓服務能夠同時應對大量使用者。

2-2 如果不用 KV Cache 會產生什麼問題？



- **回應速度**：生成每個字的時間從毫秒級暴增至數秒，使用者體感上就是 AI「打字」極度卡頓。
- **延遲倍數**：對話越長，差距越大；長對話場景下，無快取版本的等待時間可輕易達到有快取版本的數十倍。
- **資源消耗**：每一輪都做全量計算，GPU 的負擔持續累積，同時服務多位使用者時系統極易過載。

● 時間複雜度：

無 KV Cache 時生成第 k 個 Token 時，需要拿 Query 去比對前面所有 $k-1$ 個 Token 的 Key，運算量為 $O(k \cdot d)$ 。從第 1 個 Token 生成到第 n 個，累加起來就是 $O(1 \cdot d) + O(2 \cdot d) + \dots + O(n \cdot d) = O(n^2 \cdot d)$

有 KV Cache：生成第 k 個 Token 時，舊的 K、V 直接從快取讀取，只需計算新 Token 自己的

K、V 並做一次比對，每步固定為 $O(d)$ 。生成 n 個 Token 的總運算量降為 $O(n \cdot d)$ 。

● 空間複雜度：

無 KV Cache：不需要保存過去的 K、V，記憶體用量極小，僅需儲存當前步驟的中間結果。

有 KV Cache：每一層（共 L 層）都需要儲存 n 個 Token 的 K 矩陣與 V 矩陣，空間需求為 $O(n \cdot L \cdot d)$ 。

以實際模型為例，GPT 系列模型通常有 96 層、 d 達到 12,288 維，長對話下快取佔用的 VRAM 可達數十 GB，這也是為什麼記憶體容量是限制長對話上限的關鍵因素。

KV Cache 就是一種典型的「以空間換時間」策略——犧牲記憶體來換取時間複雜度從 $O(n^2)$ 降為 $O(n)$ ，讓長對話的回應速度維持穩定，不會隨對話長度爆炸性增長。

3. VRAM Compression and Trade-off

KV Cache 雖然解決了計算重複的問題，但它把所有歷史 K、V 矩陣常駐在 GPU 記憶體中，當對話越來越長、或是多位用戶同時使用時，VRAM 消耗會非常驚人。

3-1 VRAM 消耗

以一個 70B 參數的 LLM 模型，如 LLaMA-3 70B 為例，處理 4096 個 Token 的對話時，KV Cache 大約佔用 20–30 GB 的 VRAM。一張高階 GPU（如 NVIDIA A100）只有 80 GB 的 VRAM，扣掉模型本身的參數就所剩無幾，無法服務多位使用者同時對話。這就是為什麼需要壓縮技術。

3-2 VRAM Compression 技術說明

A. 4-bit 量化 (Quantization)

原本每個 K、V 數值用 16 個位元 (FP16) 儲存，量化後改用 4 個位元 (INT4) 儲存，記憶體用量直接縮小 4 倍。代價是數值精度稍微降低，AI 偶爾在細節描述上可能出現輕微偏差，但整體回答品質在大多數場景下仍可接受。

B. Paged Attention (vLLM 框架)

技術靈感來自作業系統的「虛擬記憶體分頁」，傳統 KV Cache 需要一塊連續的 VRAM 空間，很容易產生記憶體碎片。Paged Attention 把 KV Cache 切成小頁 (Block)，像管理記憶體一樣動態分配，讓多位使用者可以更有效率地共享同一塊 GPU 記憶體，GPU 吞吐量最高可提升 24 倍。

C. Sparse Attention

在一段很長的對話中，並不是所有舊的 K、V 都同樣重要。Sparse Attention 只保留「重要性高」的 K、V（例如最近的幾句話，以及全文中最關鍵的幾句），捨棄低相關性的快取。這大幅節省了記憶體，但代價是 AI 可能在極長的對話中，對很久以前說過的細節有所遺忘。

壓縮技術	優點	代價
4-bit 量化 (Quantization)	將 FP16 權重壓縮為 INT4，大幅縮小 KV Cache 體積	精度略降，部分回答可能稍有偏差
Paged Attention (vLLM)	像作業系統分頁管理記憶體，讓多用戶共享 GPU 記憶體	實作複雜度高，需要框架層級支援
Sparse Attention	只保留重要的 KV 對，丟棄低相關性的快取	可能遺失少量遠距上下文資訊

表 1：VRAM 壓縮技術比較

核心取捨

這三種技術都是在「記憶體用量 vs. 模型精度 vs. 實作複雜度」之間做取捨。

Quantization 最易部署但犧牲精度；

Paged Attention 提升 throughput 但需要框架層整合；

Sparse Attention 節省空間但可能影響長程上下文理解。

因此實際部署時往往會將多種技術組合使用。

舉例高併發雲端 API 服務配置，需要同時解決「記憶體碎片」與「單一快取體積過大」的雙重瓶頸。系統底層採用 Paged Attention，將 VRAM 切成小分頁（Block）進行動態分配，消除記憶體碎片並允許用戶間高效共享記憶體。同時寫入這些分頁的 K、V 矩陣資料，會先透過 4-bit 量化將數值從 FP16 壓縮為 INT4 格式。Paged Attention 大幅提升了系統的併發吞吐量，而量化技術則讓每個分頁內實際佔用的 VRAM 體積直接縮小 4 倍，讓同一張顯示卡能塞入更多用戶的分頁。此組合在現代推論框架（如 vLLM）中已高度整合，雖然 4-bit 量化會帶來極微幅的解題精度下降，但它用最低的實作門檻，換取了最大化的 VRAM 節省與伺服器承載量。

4. Conclusion

KV Cache 是現代 LLM 推論不可或缺的基礎技術，它將 Dynamic Programming 中 Memoization 的核心思想轉化為解決 Transformer 運算瓶頸的利器，使時間複雜度成功由 $O(n^2)$ 降至線性 $O(n)$ ，奠定了現代流暢對話 AI 的技術基石。

然而，這項「以空間換時間」的策略也導致 VRAM 的瓶頸，面對長文本與高併發的工業界需求，現代推論優化已不再局限於單一策略，而是走向如「Paged Attention 結合量化」這類硬體、框架與演算法三方協同的複合型架構，在系統層級消除碎片的同時在資料層級壓縮體積。從一個簡單的暫存想法，演進為當前大模型部署的工程，KV Cache 展現了如何透過「軟體演算法的取捨」來突破「硬體物理限制」的精彩實踐。

5. References

1. Vaswani, A. et al. (2017). Attention is All You Need. NeurIPS.
2. Kwon, W. et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP.
3. Frantar, E. et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323.
4. Child, R. et al. (2019). Generating Long Sequences with Sparse Transformers. arXiv:1904.10509.
5. Anthropic (2024). Claude Technical Report. anthropic.com.